That's it! Now, let's use this library in a sample program. It's better to create a header file declaring the functions defined in the library—*foobar.h:*

```
#ifndef _FOO_BAR_
#define _FOO_BAR_
void foo(int *);
void bar(int *);
#endif
```

Here is a sample program *(test.c)* that uses the static library:

```
#include<stdio.h>
#include"foobar.h"
int main(int argc, char **argv){
        int i=5;
        int j=10;
        foo(&i);
        bar(&j);
        printf("i = %d\n",i);
        printf("j = %d\n",j);
        return 0;
}
```

Let's compile the sample program with *gcc test.c -L./ -lfoobar;* the '-L' option is for the library path, and '-l' for the library name. GCC does not require the complete name of the library file; we can omit the prefix *'lib'* and the extension *'.a'* or *'.so'* (in case of a shared library), so let's use *-lfoobar.* This command yields the output file *'a.out'*, which will have the contents of the static library compiled into the binary. You can even delete the *libfoobar.a* file now, and you can still run the output file:

```
$ ./a.out
i = 10
j = 50
```

So we have now built a static library with *ar,* and used it, statically compiled into a program.

## The 'ld' tool is important

*Linking* is the process of combining various pieces of code and data together to form a single executable image (that can be loaded) in memory. Linking can be done at compile time or run-time. GNU GCC performs linking in the background; compile with the *'-v'* option, and you will see many background details, including the linking. To understand what happened during compilation, you must know how to use a linker manually—so let's compile a simple C 'Hello World' program without linking it (as before, *'-c'*) with *gcc -c hello.c.* Let's manually do linking of the resultant object file *'hello.o'.* First, you must know that *main()* is the starting point of the program. Now, to make an executable, add some more object code, and the 'c' library, into the final executable, with the following command:

```
$  ld -o hello -dynamic-linker /lib/ld-linux.so.2 /usr/lib/
crt*.o hello.o -lc
```

Here, *'-o'* names the output file *'hello'; '-dynamic-linker'* will include the shared library symbols from *'ld-linux.so.2'* in the executable; we will include *hello.o* and the other object files required, and finally include the C static library with *'-lc'.* In the output executable *'hello', /usr/lib/crt*.o, hello.o,* and the C library *(-lc)* are statically linked to (copied into) *'hello',* and */lib/ld-linux.so.2* are linked dynamically.

How are *'ar'* and *'ld'* different? The *'ar'* tool can only archive binary files statically, whereas *'ld'* links both shared and static libraries. Also, *'ld'* resolves symbols, which *'ar'* doesn't.

We don't need to use *'ld'* manually, since GCC handles this—but to learn about what is in the background, we tried the above steps.

## NM

Figure 1 represents the memory segments of a C program (heap, data, code and stack), which C programmers must consider. Different variables and symbols make their entry into different sections: dynamically allocated memory in heap, static and global variables in the data section; code and constants in the text part; and local variables in the stack section. It's hard to investigate which symbol goes to which section, so many thanks to open source developers, for the amazing tool *'nm',* which can dissect 'a.out' and detail the symbols present in it. Let's try this with a sample program, *test.c:*

```
#include<stdio.h>
#include<stdlib.h>
#include<string.h>
/* Declare some global variables*/
int global_int1;
int global_int2=10;
char global_string[10];
const int const_int = 10;
void test1(void){
        global_int1 = 20;
        printf("[test1] global_int2 = %d\n",global_int2);
}
void test2(void){
        strcpy(global_string,"Hello");
        printf("[test2] global_int1 = %d\n",global_int1);
}
void test3(void){
```